CHARITYCHAIN
DECENTRALIZED CROWDFUNDING PLATFORM

Used technologies

Solidity

Hardhat

Ethers.js

MetaMask

Team members:
D. Dinara
J. Yerassyl
S. Zhanassyl

D. Dinara

S. Zhanassyl

J. Yerassyl

PROBLEM & GOAL

Problem

- Lack of transparency in donations
- Trust in centralized platforms

Goal

- Campaign creation and donations
- Automated reward system for donors

Key Idea

- On-chain state as a single source of truth

APPLICATION ARCHITECTURE

Blockchain Layer :

- Smart contracts store campaigns and funds

Frontend Layer :

- Browser-based user interface

Infrastructure :

- Hardhat for deployment, testing, and local blockchain

SMART CONTRACTS

CharityChain.sol

- Campaign management logic
- Donations, withdrawals, refunds

RewardToken.sol

- ERC-20 reward token (CCRT)
- Minting controlled by CharityChain

Design Choice

- Separation of responsibilities

CAMPAIGN DATA MODEL

Campaign Structure

- id, creator, title
- goal (wei), deadline
- totalRaised
- finalized, successful

Mappings

- User contributions per campaign
- Refund status tracking

CAMPAIGN LIFECYCLE

Step 1
Campaign creation by creator

Step 2
ETH contributions from users

Step 3
Campaign finalization

Campaign finalization

Outcome

- Success -> creator withdraws funds
- Failure -> contributors request refunds

REWARD TOKEN MECHANISM

Token
ERC-20 CharityChain Reward (CCRT)

Reward Logic
1000 CCRT per 1 ETH

Minting
Tokens minted during contribution

Only CharityChain can mint

REFUND LOGIC

When Refunds Are Allowed

Campaign finalized

Goal not reached

Conditions

User contributed ETH

Refund not claimed before

Guarantee

No double refunds

SECURITY MEASURES

Reentrancy Protection

ReentrancyGuard on ETH transfers

State Safety

State updated before transfers

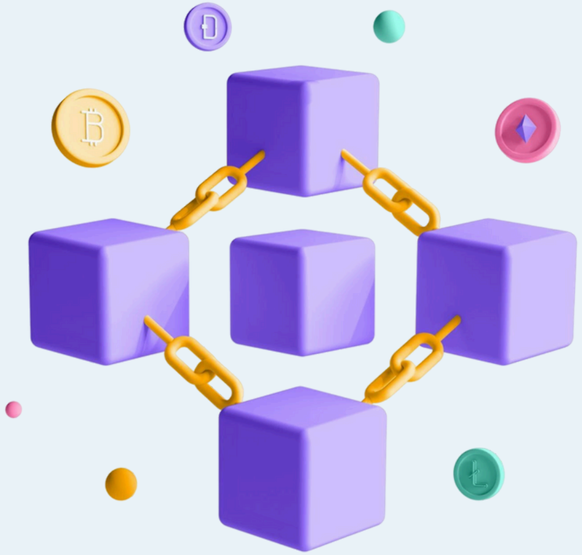
Access Control

Only campaign creator can withdraw

FRONTEND & BLOCKCHAIN INTERACTION

Wallet METAMASK

MetaMask for account access and signing



Library

Ethers.js for contract interaction

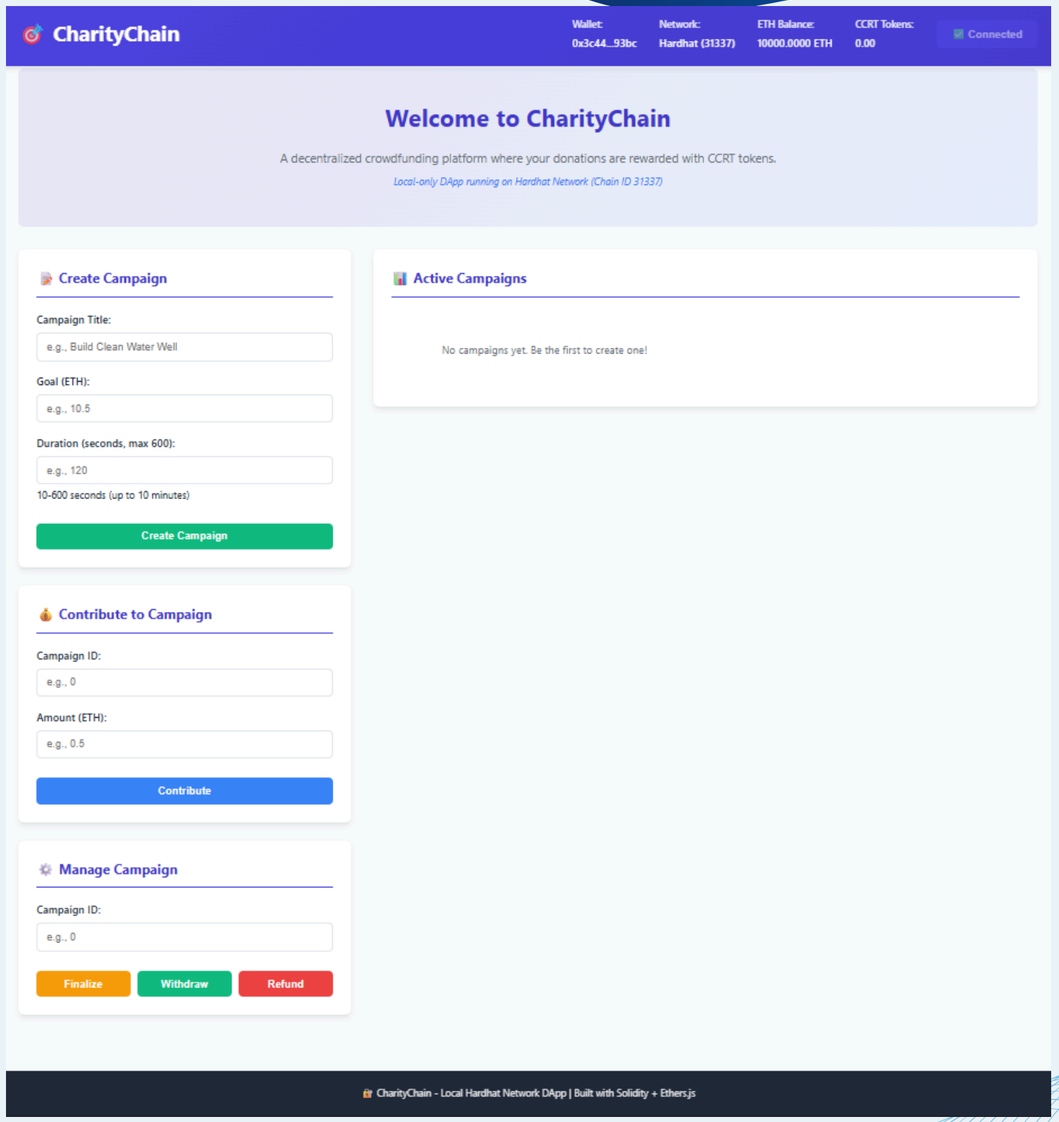
Network

Hardhat Local Network (Chain ID: 31337)

USER INTERFACE FEATURES

Core Actions:

- Create campaign
- Donate ETH
- Finalize campaign
- Withdraw or refund
- Visualization
- Campaign cards
- Progress bars
- Status indicators



DEPLOYMENT PROCESS

1. Local Blockchain
2. Hardhat network
3. Deployment
4. Deploy smart contracts
5. Set minter role
6. Generate frontend config
7. Execution
8. Run frontend and connect MetaMask

TESTING

- Test Framework
- Mocha and Chai
- Test Coverage
- Campaign creation
- Contributions and rewards
- Finalization logic
- Withdrawals and refunds
- Access control

CONCLUSION

What the Project Demonstrates:

- Full decentralized application workflow
- Smart contract security and logic
- Frontend-to-blockchain integration

Result - transparent and trustless crowdfunding system

THANK YOU